// ALGORITHMIC TRADING SOCIETY PRESENTS

ALL IN

THE ALGOSOC POKERBOT COMPETITION

RULES & BOT API HANDBOOK

DAY 1 · GROUPS	DAY 2 · KNOCKOUTS	VENUE
WEDNESDAY 25 NOVEMBER 2026	**THURSDAY 26 NOVEMBER 2026**	**HUXLEY BUILDING, SOUTH KENSINGTON**

IMPERIAL COLLEGE LONDON

ALGOSOC.COM

Contents

Everything a team needs for **All In**: how the event runs, exactly how the poker works, how you are scored, and how to write and submit a bot. Part I is the official rulebook. Part II is the technical reference for the bot interface.

1. At a glance 4
2. Eligibility and teams 4
3. The event 4
4. Building and submitting your bot 5
5. The format 6
6. Betting rules 7
7. Scoring 10
8. Prizes 11
9. Conduct and disputes 11
10. Quick start 14
11. How your bot is run 14
12. The state object 15
13. Returning an action 17
14. Hook payloads 17
15. Limits and the sandbox 18
16. Submitting 19
17. Common mistakes 19
Appendix: questions 19

SETTINGS IN FORCE

This handbook was generated from the platform's live settings: **No-Limit Hold'em**, no raise cap, **10 ms** per decision, **3** group rounds of 200 sit-and-gos, 500 sit-and-gos per knockout table and 250 duplicate deals per heads-up match. If the organisers change a setting, the website's Rules page is always the current version.

PART I

THE RULES

How the event runs, how the poker works and how you are scored.



1. At a glance

The whole competition on one page. Every line is explained in detail in the sections that follow.

TOPIC	RULE
Game	No-Limit Texas Hold'em, no cap on raises
Table	Eight-bot sit-and-go (tournament style: play until one bot has every chip)
Starting stack	2,000 chips per bot, every table
Blinds	10/20, rising every 25 hands
Hand cap	200 hands; survivors split the remaining points by ICM
Points	1st: 100 · 2nd: 60 · 3rd: 40 · 4th: 25 · 5th: 15 · 6th: 10 · 7th: 5 · 8th: 0
Format	Two days. Day 1: groups of up to 8, 3 group rounds. Day 2: Round of 16 and quarter-final (8-bot tables), then heads-up semi-finals, final and 3rd-place match
Day 1 score	G1 + 2×G2 + 3×G3; each round = average points per game
Qualification	Top of each group + best of the rest = 16, seeded by Day 1 score
Decision clock	10 ms per decision; timeouts and errors check or fold
Language	Python 3, one Agent class, no network access during play
Submissions	bot.py or a .zip (max 100 MB); one upload every 5 minutes
Teams	One or two Imperial students

2. Eligibility and teams

2.1 All In is open to current Imperial College London students who register on the platform with an Imperial email address (@imperial.ac.uk, @ic.ac.uk).

2.2 Teams have **one or two members**. Each person registers individually, then one member creates the team and shares its six-character **join code**; the other joins with it. You may be in only one team.

2.3 Teammates don't need to be on the same course or in the same year.

2.4 AlgoSoc committee members who run the event may not compete for prizes.

2.5 A member can leave a team from the team page. A team with no members and no results is deleted along with its submissions.

3. The event

3.1 All In runs over **two days** at Huxley Building, South Kensington: **Day 1** on Wednesday 25 November 2026, 18:00 – 21:30, and **Day 2** on Thursday 26 November 2026, 18:00 – 21:30.

3.2 The schedule:

WHEN	STAGE	WHAT HAPPENS
Day 1	Build phase (60 min)	A countdown on the big screen. Download the starter kit, write your bot, test it locally and upload it.
Day 1	G1–G3	Group rounds. Each team plays 200 sit-and-gos per round against its own group.
Overnight	—	Qualifiers are announced. Uploads stay open; keep improving your bot.
Day 2	R16	Two 8-bot tables of 8 qualifiers, 500 sit-and-gos each. Top 4 per table go through.
Day 2	QF	One 8-bot table, 500 sit-and-gos. Top 4 go through.
Day 2	SF	Heads-up: 1st v 4th and 2nd v 3rd from the quarter-final.
Day 2	F	Heads-up final, and a heads-up match for 3rd place.

3.3 The organisers announce each stage with a countdown on the big screen (`/live` on the platform). A stage usually takes a few minutes to run.

3.4 Groups, the bracket, every stage's results and table replays are public on the platform's Competition page.

4. Building and submitting your bot

4.1 Bots are written in **Python 3** and implement the `Agent` class described in Part II. Use the starter kit from the Bot API page.

4.2 Upload either a single `bot.py` or a `.zip` of up to **100 MB** with `bot.py` at its top level (or inside one top-level folder).

4.3 Each team may upload **once every 5 minutes**.

4.4 Every upload is immediately tested on short tables against house bots. If it loads and makes decisions, it **passes** and becomes your **active bot**. If it can't be imported, has no `Agent` class, or every decision fails, it is rejected and your previous active bot stays in place.

4.5 Your team page shows each upload's test report: decisions made, average and slowest decision time, timeouts, errors with tracebacks, and anything your bot printed.

4.6 The **most recent passing upload** is always your active bot. Only the active bot plays.

4.7 Each decision must be made within **10 ms**. A bot that errors, times out or returns something invalid **checks if it can, and otherwise folds**. The table carries on.

4.8 Bots run in their own process with no network access. During play a bot may not call external APIs, AI services or any outside help. You may use any tools you like, AI included, while building.

4.9 A fresh `Agent()` is created for every table, so memory lasts for one sit-and-go only. Opponents are anonymous: you see seat numbers, never team names.

5. The format

Every game

5.1 Every game is a sit-and-go: it ends when one bot holds every chip, or after **200 hands**. Each bot starts with **2,000 chips**.

5.2 Blinds rise every **25 hands**. There are no antes. The schedule:

LEVEL	HANDS	SMALL / BIG BLIND	STARTING STACK IN BIG BLINDS
1	1–25	10 / 20	100
2	26–50	20 / 40	50
3	51–75	30 / 60	33
4	76–100	50 / 100	20
5	101–125	75 / 150	13
6	126–150	100 / 200	10
7	151–175	150 / 300	7
8	176–200	200 / 400	5

5.3 Your active bot is **frozen** the moment each stage starts. Uploads made during a stage play from the next one.

5.4 Eight-bot tables with fewer than eight teams fill the empty seats with **copies** of the table's own bots. Copies play normally but **score no points for anyone**.

Day 1: group stage

5.5 **The draw.** When Group round 1 starts (or earlier, if the organisers announce the groups), every team with an active bot is drawn at random into groups of at most eight, as evenly as possible. Example: 20 teams make groups of 7, 7 and 6. Teams with no active bot at that moment miss the group stage.

5.6 **Group rounds.** There are 3 rounds. In each round, every group plays 200 sit-and-gos among its own members, with seats shuffled every game.

5.7 **Day 1 score.** Round r counts $\times r$, so later rounds matter more:

```
DAY 1 = 1×G1 + 2×G2 + 3×G3
```

5.8 **Qualification.** 16 teams reach Day 2. With *n* groups, the top 16 ÷ *n* (rounded down) of each group qualify automatically. The remaining places go to the highest Day 1 scores among everyone else (**wildcards**):

GROUPS	AUTO-QUALIFY PER GROUP	WILDCARDS
2	8	0
3	5	1
4	4	0
5	3	1
6	2	4

5.9 The 16 qualifiers are **seeded 1–16 by Day 1 score**.

Day 2: knockouts

5.10 **Round of 16.** Seeds are snaked across two tables so the strongest teams are split: Table A gets seeds 1, 4, 5, 8, 9, 12, 13, 16 and Table B gets 2, 3, 6, 7, 10, 11, 14, 15. Each table plays 500 sit-and-gos. The **top 4** at each table go through.

5.11 **Quarter-final.** The last eight play one table of 500 sit-and-gos. The **top 4** go through, ranked 1–4 by quarter-final score.

5.12 **Semi-finals.** Heads-up: quarter-final 1st v 4th, and 2nd v 3rd. Winners play the final; losers play the 3rd-place match.

5.13 **Heads-up matches** are played in **duplicate**: 250 deals, each played twice with the bots swapping seats, so both bots get exactly the same cards. Every game is a heads-up sit-and-go with the usual stacks, blinds and hand cap. A game won scores 1; at the hand cap the game is split by chips (ICM). The bot with the higher share of games wins the match.

5.14 If a knockout opponent is missing, the other team goes through on a walkover.

5.15 If a stage fails or is disrupted, the organisers may re-run it. Re-running a stage replaces its results and those of every later stage.

WHY DUPLICATE?

In heads-up, one lucky run of cards can decide a short match. Playing every deal from both seats cancels the cards out, so the better bot wins. Two identical bots always split exactly 50/50.

6. Betting rules

All In is played as **No-Limit Texas Hold'em**. This section explains exactly what a bot may do and how every situation is resolved.

STRUCTURE	MINIMUM RAISE	MAXIMUM RAISE	RAISES PER STREET
No-Limit	The previous bet or raise this street, never less than the big blind	Your whole stack (all in)	No cap

The hand, step by step

6.1 **Button.** The dealer button moves one live seat to the left each hand. Busted seats are skipped.

6.2 **Blinds.** The next two live seats after the button post the small and big blind automatically. Heads-up (two bots left), the button posts the small blind and the other bot the big blind.

6.3 **Deal.** Each bot gets two private hole cards. Then there are four betting rounds: **pre-flop**, **flop** (three board cards), **turn** (fourth card) and **river** (fifth card).

6.4 **Order of action.** Pre-flop, action starts with the seat left of the big blind and the big blind acts last. On the flop, turn and river, action starts with the first live seat left of the button. Heads-up, the button acts first pre-flop and last on later streets.

6.5 **End of a betting round.** A round ends when every bot still in the hand has matched the highest bet, folded or gone all in, and everyone has acted since the last full raise. If all but one bot fold, that bot wins the pot immediately and no cards are shown.

6.6 **Run-out.** If at most one bot can still bet (the rest are all in), the remaining board cards are dealt with no more betting.

Your options

ACTION	WHEN IT'S ALLOWED	WHAT IT DOES
`fold`	Facing a bet	Give up the hand and any chips already put in.
`check`	Nothing to call	Pass without betting.
`call`	Facing a bet	Match the current highest bet (or go all in if you can't match it).
`raise`	When raising is open	Bet or raise **to** a total for this street, within `min_raise_to`…`max_raise_to`. A first bet on a street is also called a raise.
`all_in`	When raising is open	Raise to the largest amount allowed (your whole stack in No-Limit).

6.7 **Raise to, not raise by.** Amounts are the total you want in front of you on this street. If the big blind is 40 and you want to make it 120, return `{"action": "raise", "amount": 120}`, not 80.

6.8 **Minimum raise.** A raise must increase the bet by at least the size of the previous bet or raise on the same street, and never by less than the big blind.

WORKED EXAMPLE: MINIMUM RAISES

Blinds 20/40. Seat 3 raises to 120 (an increase of 80). The next raise must be to at least **200** (120 + 80). If seat 5 then raises to 300 (an increase of 180), the next raise must be to at least **480**. On a new street, the minimum bet goes back to one big blind (40).

6.9 **Maximum raise.** There is no maximum other than your stack. You may go all in at any time.

6.10 **Raise cap.** There is no limit on the number of raises per street.

All-ins, stakes and side pots

6.11 **Table stakes.** You can only bet the chips in front of you at the start of the hand. There are no rebuys, add-ons or loans. A bot with no chips left is eliminated from that table.

6.12 **Short all-ins.** You may always go all in, even for less than a full raise or less than a call. A short stack posts whatever it has as a blind.

6.13 **Incomplete raises.** An all-in that raises by less than a full minimum raise does not reopen the betting. Bots that have already acted since the last full raise may call or fold, but not re-raise. Bots that haven't acted yet may still raise.

6.14 **Uncalled bets.** Any part of a bet that nobody can call is returned to its owner before the pot is awarded.

6.15 **Side pots.** When a bot is all in for less than others have bet, it can only win the chips it matched from each opponent. The rest forms a side pot that only the bots who paid into it can win.

WORKED EXAMPLE: SIDE POTS

Bot A is all in for 300. Bot B is all in for 800. Bot C calls 800. **Main pot**: 300 from each = 900, which A, B and C can win. **Side pot**: the extra 500 from B and C = 1,000, which only B and C can win. If A has the best hand, A wins 900, and the best of B and C wins the 1,000.

Showdown and hand rankings

6.16 If two or more bots remain after the river betting, all their hands are shown. Each bot uses the best five-card hand from its two hole cards and the five board cards, in any combination.

6.17 Each pot (main and side) goes to the best hand among the bots eligible for it. Suits have no rank. Tied hands split the pot evenly; any odd chip goes to the first tied winner left of the button.

RANK	HAND	EXAMPLE	TIE-BREAK
1	Straight flush	9h 8h 7h 6h 5h	Highest card (A-K-Q-J-T is the best; A-2-3-4-5 the lowest)
2	Four of a kind	Qs Qh Qd Qc 4s	Rank of the four, then kicker
3	Full house	Js Jd Jc 7h 7s	Rank of the three, then the pair
4	Flush	Ad Jd 8d 6d 2d	Highest card, then next, and so on
5	Straight	Ts 9d 8c 7h 6s	Highest card (ace can be low in A-2-3-4-5)
6	Three of a kind	8s 8h 8d Kc 3s	Rank of the three, then kickers
7	Two pair	Ks Kd 5c 5h Ts	Higher pair, lower pair, kicker
8	One pair	As Ad Jc 9h 4s	Pair, then three kickers
9	High card	Ah Qd 9c 6s 3h	Highest card, then next, and so on

Illegal, invalid and missing actions

The dealer never stops the game to reject a decision. It corrects it as follows:

YOUR BOT RETURNS…	RESULT
A raise amount below the minimum or above the maximum	Moved to the nearest legal amount
A raise amount that isn't a number	Treated as a minimum raise
raise when raising isn't allowed (cap reached, betting not reopened, or no one left to bet against)	Call (or check if there is nothing to call)
call with nothing to call	Check
fold with nothing to call	Check (folding for free is never useful)
check when facing a bet	Fold
An unknown action, None, or anything unreadable	Check if possible, otherwise fold
An exception, or no answer within 10 ms	Check if possible, otherwise fold (counted as an error or timeout)
Bot fails to load at a table	Every decision at that table is check/fold

7. Scoring

7.1 At each table, bots are paid points for their finishing place:

1ST	2ND	3RD	4TH	5TH	6TH	7TH	8TH
100	60	40	25	15	10	5	0

7.2 **Eliminations.** A bot that loses all its chips takes the best place still open. Example: the first bot out of eight finishes 8th.

7.3 **Several busts in one hand.** If bots are knocked out in the same hand, the one with the bigger stack at the start of that hand finishes higher. Exact ties are decided at random.

7.4 **The hand cap and ICM.** If 200 hands are played and several bots still have chips, the points for the places still open are shared using the **Independent Chip Model** (ICM). ICM gives each survivor its expected points if the table were played out, assuming each bot's chance of finishing first equals its share of the chips, and so on down the places.

WORKED EXAMPLE: ICM AT THE HAND CAP

Three bots survive the cap with 8,000, 5,000, 3,000 chips. The open places pay 100, 60, 40 points. ICM gives them **76.85, 66.44, 56.70** points. The chip leader gets the most, but less than a straight share of the chips would suggest, because first place pays only a little more than the others.

7.5 **Stage score.** In every stage, a team's score is its average points per game. In heads-up matches that is the share of games won.

7.6 **Ranking** inside a group, table or match: higher average points, then more outright wins (1st places), then more chips on average at the end, then the better seed.

WORKED EXAMPLE: DAY 1 SCORE

A team averages 30, 36 and 42 points in G1, G2 and G3. Its Day 1 score is 30 + 2×36 + 3×42 = **228**. G3 alone is more than half of it.

7.7 **Final standings.** 1st and 2nd from the final; 3rd and 4th from the 3rd-place match; 5th–8th by quarter-final rank; 9th–16th by round-of-16 rank, then score; everyone else by Day 1 score.

7.8 A team with no active bot when a group round starts scores 0 for that round.

7.9 Chips exist only inside the simulation and have no cash value.

8. Prizes

PLACE	PRIZE
1st	£1,000
2nd	£500
3rd	£250

8.1 Prizes are awarded per team and split evenly between teammates.

8.2 Before prizes are awarded, the organisers may ask any team to show and explain its code.

9. Conduct and disputes

9.1 Teams are disqualified for:

– colluding with another team, or sharing code between teams;

- copying another team's bot;
- trying to identify, signal to or coordinate with other bots at the table (for example soft-play patterns between teammates' bots);
- interfering with the platform, the host machine or other bots: reading other submissions, exhausting resources on purpose, network access, or attacking the sandbox;
- submitting malware, credentials or personal data.

9.2 Each team may only have one active bot. Making several teams to field extra bots is collusion.

9.3 Raise disputes with the organisers before the next stage starts. For the final, within five minutes of results being published. Hand logs are kept to settle disputes.

9.4 The organisers may update these rules, settings or schedule during the event if needed, and will announce any change before the next stage. The organisers' decision is final.

PART II

THE BOT API

Everything your code sees, everything it may return, and how it is run.



10. Quick start

Download the starter kit from the platform's Bot API page. It contains:

FILE	WHAT IT IS
bot.py	Your bot. A complete, working template; edit this and upload it.
local_sim.py	Plays your bot against others on your machine, under the real time limit.
examples/	House bots: tight_aggressive.py, calling_station.py, random_bot.py.
engine/	The exact engine the platform runs (cards, game, ICM, bot runner).
README.md	A short summary of this handbook.

The smallest valid bot checks when it can and folds otherwise:

```
class Agent:
    def act(self, state):
        if "check" in state["legal_actions"]:
            return {"action": "check"}
        return {"action": "fold"}
```

Then test it locally and upload `bot.py` from your team page:

```
python local_sim.py                        # bot.py vs house bots, 50 tables
python local_sim.py --tables 200 --timeout 10
python local_sim.py bot.py examples/tight_aggressive.py
python local_sim.py --betting no_limit
```

11. How your bot is run

STEP	WHAT HAPPENS
1. Load	For each table, the platform starts a new Python process and imports your bot.py. Your folder is the working directory, so you can open files that ship with your zip.
2. Construct	It calls Agent() with no arguments, then on_table_start(info) if you define it. You have 5 seconds for import plus setup.
3. Decide	Whenever it's your turn, it calls act(state). You must return within 10 ms.
4. Learn	After every hand you took part in, it calls on_hand_end(summary) if you define it. Not timed, but keep it quick.
5. Finish	When the table ends, the process is stopped. Nothing carries over to the next table.

The full class, with every optional hook:

```
class Agent:
    def __init__(self):
        # Runs once per table. Load lookup tables, seed RNGs, etc.
        self.hands_seen = 0
    def on_table_start(self, info):          # optional
        self.seat = info["seat"]
    def act(self, state):                    # required, timed
        return {"action": "call"}
    def on_hand_end(self, summary):          # optional, not timed
        self.hands_seen += 1
```

12. The state object

`act` receives a plain dict. Seats are numbered 0–7, and every list is indexed by seat. Cards are two-character strings: rank `23456789TJQKA` then suit `c d h s`, e.g. `"As"` is the ace of spades and `"Td"` the ten of diamonds.

A real example

Here is a real state captured from the engine on the flop, facing a bet. Every field is explained in the table below.

```
{
  "hand_number": 2,
  "street": "flop",
  "seat": 0,
  "button": 0,
  "hole_cards": [ "Kh", "7c" ],
  "board": [ "3h", "6s", "6h" ],
  "small_blind": 10,
  "big_blind": 20,
  "blind_level": 0,
  "pot": 425,
  "stacks": [ 2010, 1970, 1910, 1765, 2000, 2000, 1920, 2000 ],
  "bets": [ 0, 0, 0, 155, 0, 0, 0, 0 ],
  "folded": [ false, true, true, false, true, true, true, true ],
  "to_call": 155,
  "min_raise_to": 310,
  "max_raise_to": 2010,
  "legal_actions": [ "fold", "call", "raise" ],
  "betting": "no_limit",
  "raise_cap": null,
  "raises_this_street": 1,
  "history": [   // first 3 of 13 entries
    {"seat": 1, "street": "preflop", "action": "small_blind", "amount": 10},
    {"seat": 2, "street": "preflop", "action": "big_blind", "amount": 20},
    {"seat": 3, "street": "preflop", "action": "call", "amount": 20}
  ]
}
```

Every field

KEY	TYPE	MEANING
seat	int	Your seat number, 0–7.
hole_cards	list[str]	Your two private cards.
board	list[str]	Community cards so far: 0, 3, 4 or 5 cards.
street	str	"preflop", "flop", "turn" or "river".
hand_number	int	1 to 200 at this table.
button	int	Seat of the dealer button.
small_blind, big_blind	int	Blinds this hand.
blind_level	int	0-based blind level.
pot	int	All chips put in this hand, including bets on the current street.
stacks	list[int]	Chips each seat has behind. 0 means busted, or all in this hand.
bets	list[int]	Chips each seat has put in on the current street.
folded	list[bool]	True if a seat is out of this hand. Busted seats count as folded.
to_call	int	Chips you need to call, capped at your stack. 0 means you can check.
min_raise_to	int | None	Smallest legal raise, as a total for this street. None if you can't raise.
max_raise_to	int | None	Largest legal raise, as a total for this street. None if you can't raise.
legal_actions	list[str]	Some of "fold", "check", "call", "raise".
betting	str	"no_limit", "pot_limit" or "fixed_limit".
raise_cap	int | None	Max bets/raises per street; None means no cap.
raises_this_street	int	Bets/raises made so far on this street (blinds not counted).
history	list[dict]	Every action this hand, in order: {seat, street, action, amount}. Actions include small_blind, big_blind, fold, check, call (amount added) and raise (raise-to total).

13. Returning an action

Return any of these forms. Dicts are clearest:

```
{"action": "fold"}
{"action": "check"}
{"action": "call"}
{"action": "raise", "amount": 200}     # raise TO 200 on this street
{"action": "all_in"}                    # largest raise allowed
("raise", 200)                          # tuples work too
"call"                                  # so do bare strings
```

Anything outside the rules is corrected as described in the table in section 6. Useful patterns:

```
def act(self, state):
    legal = state["legal_actions"]
    if "raise" in legal:
        pot_bet = state["bets"][state["seat"]] + state["to_call"] + state["pot"]
        size = max(state["min_raise_to"], min(pot_bet, state["max_raise_to"]))
        return {"action": "raise", "amount": size}
    if "check" in legal:
        return {"action": "check"}
    if state["to_call"] <= 2 * state["big_blind"]:
        return {"action": "call"}
    return {"action": "fold"}
```

14. Hook payloads

on_table_start(info)

KEY	MEANING
seat, num_players	Your seat and the table size (8).
starting_stack	Chips each bot starts with.
blinds, hands_per_level	The full blind schedule as [[sb, bb], ...] and how often it rises.
max_hands	The hand cap.
points	Points for each finishing place, 1st first.
betting, raise_cap	Betting structure and raise cap (None = no cap).

on_hand_end(summary)

KEY	MEANING
hand_number	The hand that just finished.
board	Final board (may be short if everyone folded).
shown	Hole cards revealed at showdown, as {"seat": [cards]}. Empty if there was no showdown.
winners	List of {seat, amount, hand}; hand is e.g. "Two Pair", or None if uncontested.
stacks	Every seat's chips after the hand.
history	Every action in the hand.

USE IT

on_hand_end is the only time you see opponents' cards. Combine `shown` with `history` to learn how each seat bets with different hands. Remember seats are anonymous and memory resets every table.

15. Limits and the sandbox

LIMIT	VALUE
Decision time	10 ms per act() call, measured by the dealer (includes the time to pass the state to your process)
Setup time	5 s per table for import, Agent() and on_table_start
CPU	One process per seat; 300 CPU-seconds per table. Use a single thread.
Network	None during play
Upload size	100 MB (zip); unpacked contents up to 300 MB
Libraries	Python standard library. Ask the organisers before relying on numpy or other packages.
Output	print() is safe: it goes to a log shown in your test report, never to the dealer
Errors	Exceptions in act() are caught; that decision becomes check/fold and is counted as an error

TIME BUDGET

Python itself takes a fraction of a millisecond per call, so plan for about 10 ms of real thinking at most. Precompute heavy tables in __init__ or at import time, and use your test report's average and slowest decision times to check your margin.

16. Submitting

- Sign in, open **My Team**, drop in `bot.py` or a zip, and press **Upload & test**.
- The upload plays short test tables straight away. The page refreshes when the test finishes.
- **Passed** uploads become your **Active** bot at once. **Failed** ones show why, and your previous active bot keeps playing.
- Open the test report to see decisions, timing, timeouts, errors with tracebacks, and your bot's printed output.
- After each stage your team page shows your average points, rank, timeouts and errors, with a replay of one of your tables.

17. Common mistakes

MISTAKE	FIX
Returning the raise *increase* instead of the total	amount is raise-to: the total in front of you on this street.
Using min_raise_to when raising isn't legal	It is None then. Check "raise" in legal_actions first.
Assuming you're seat 0, or that seats are teams	Read state["seat"]. Seats are anonymous and reshuffled every table.
Expecting memory across tables	Each table gets a new process and a new Agent().
Heavy work inside act()	Precompute in __init__; watch your slowest decision time.
Forgetting the blinds rise	Use big_blind and stacks to think in big blinds. Short stacks should push or fold.
Ignoring the hand cap	Near hand 200, survivors are paid by ICM: protecting a big stack is worth more than chips.

Appendix: questions

Do I need to be good at poker?

No. Bots built on probabilities and careful testing do well. Start from the examples and measure every change with the simulator.

Can teammates upload separately?

Yes. Both members share one team page, one cooldown and one active bot.

What if my bot crashes mid-stage?

Just that decision becomes check/fold. Errors are counted and shown on your team page.

Can I see other bots' code?

No. You can watch public replays, with every hand from every stage.

Is the platform's engine the same as the starter kit's?

Yes, byte for byte. Results differ only through card luck and opponents.